LLaMA-2-7B Compression Pipeline for Edge Deployment: Structured Pruning, SmoothQuant, and Mixed-Precision Quantization

Yamsheen Saqib 26100379
Talha Asif 24280053
Syed Haider 24280049
Zain Ahmad 24280080
Lahore University of Management Sciences (LUMS)

Abstract

Large Language Models (LLMs) have revolutionized natural language processing, but their substantial memory requirements create significant barriers to deployment on resource-constrained edge devices. This project presents a comprehensive compression pipeline for LLaMA-2-7B, achieving a reduction from 13.5 GB to 3.29 GB, a 75.6% compression ratio, while preserving coherent text generation capabilities. Our approach combines four complementary techniques: activation-based structured pruning that removes 20% of feed-forward network neurons, SmoothQuant for outlier suppression enabling more aggressive quantization, a custom sensitivity analysis framework for mixed-precision bit-width allocation, and optimized GGUF conversion for efficient CPU inference. The resulting model successfully operates within the 8GB RAM constraint of a Raspberry Pi 4, demonstrating the practical viability of deploying billion-parameter language models on edge hardware without requiring specialized accelerators.

1 Introduction

1.1 Background and Motivation

Introduction of Large Language Models has completely changed the situation in the field of artificial intelligence applications. Models like GPT, LLaMA, and their variants are seen to perform exceptionally well on a variety of tasks such as text generation tasks, question answering tasks, summarisation tasks, and code completion tasks. However, these abilities are very expensive in terms of computation. Even small models like LLaMA-2-7B, which is considered small by modern standards, require around 13.5GB of memory in half-precision floating-point format (FP16), which is impractical to execute on an edge computer running on a consumer grade.

The motivation for edge deployment extends beyond mere technical curiosity. On-device inference is of significant value to privacy sensitive applications since data of users are not sent to the outside world. Latency sensitive systems do not suffer the vagaries of delays caused by network round trip in the cloud servers. In addition, edge deployment brings democratization of AI functionality, meaning that it does not rely on an expensive infrastructure of cloud services, and it can operate offline in areas with connectivity restrictions.

The Raspberry Pi 4, with its 8GB RAM configuration, is a perfect target platform for this study. It is ubiquitous and inexpensive and is illustrative of the memory limitations experienced in many

Foundation Models for Science Workshop,38th Conference on Neural Information Processing Systems (NeurIPS 2024).

edge-computing applications such as Internet-of-Things devices, embedded systems and mobile platforms.

1.2 Problem Statement

The central challenge addressed in this project is straightforward yet technically demanding: compress LLaMA-2-7B from 13.5 GB to under 4 GB while maintaining acceptable text generation quality, thereby enabling deployment on a Raspberry Pi 4 with 8GB of RAM.

1.3 Contributions

This project makes the following contributions:

Firstly, Activation-based structured pruning has been implemented and evaluated as an activation-based pruning method that deletes whole neurons of the feed-forward network layers, resulting in a 13.4% reduction in parameters with model coherence having no fine-tuning.

Secondly , we correctly implement SmoothQuant, an activation-sensitive weight-smoothing method that distributes quantisation risk by reducing curves between activations and weights; the implementation also takes into account an important detail that is frequently neglected the fact that both LayerNorm and Linear layer weights need to be scaled to maintain mathematical equivalence.

Third, we develop a custom sensitivity analysis framework that combines weight statistics and activation patterns to determine optimal bit-width allocation across different layers, providing theoretical justification for mixed-precision quantization strategies.

Fourth, we demonstrate end-to-end deployment using the GGUF format and llama.cpp inference engine, validating that the compressed model produces coherent outputs on CPU hardware.

2 Related Work

2.1 LLaMA-2 Architecture

LLaMA-2-7B follows the decoder-only transformer architecture that has become standard for autoregressive language models. The model consists of 32 identical transformer layers, each containing a multi-head self-attention mechanism and a feed-forward network (FFN). Understanding the architectural details is essential for effective compression, as different components exhibit varying sensitivity to precision reduction and structural modification.

The self-attention mechanism employs 32 attention heads, each operating on 128-dimensional subspaces of the 4096-dimensional hidden representation. The attention computation involves query (Q), key (K), and value (V) projections, each implemented as linear transformations from the hidden dimension to the concatenated head dimension.

The feed-forward network in LLaMA-2 uses the SwiGLU activation function, implemented through three linear projections: gate_proj and up_proj that expand the hidden dimension from 4096 to 11008, followed by element-wise multiplication with SiLU activation, and finally down_proj that compresses back to 4096 dimensions. This FFN structure accounts for approximately 67% of the model's total parameters, making it the primary target for compression efforts.

2.2 Model Compression Techniques

Model compression is a type of technology that is used to shrink the computational and memory needs of neural networks. The primary approaches include pruning, which removes redundant parameters or structures; quantization, which reduces the numerical precision of weights and activations; knowledge distillation, which trains smaller models to mimic larger ones; and architectural modifications such as low-rank factorization.

In edge deployment situations that are not based on training infrastructure, post-training methods are especially attractive because they do not require expensive retraining. This project focuses on structured pruning and post-training quantization, augmented with modern outlier suppression techniques.

2.3 Quantization Methods

Recent advances in quantization for LLMs include GPTQ by Frantar (2023), LLM.int8() by Dettmers (2022), AWQ by Dettmers (2022), and SmoothQuant by Lin (2023). SmoothQuant is particularly relevant as it addresses activation outliers, a key challenge in quantizing transformer models.

2.4 llama.cpp for Edge Deployment

llama.cpp is an open-source inference engine written in C/C++ that enables efficient CPU-based inference of LLaMA models. It supports various quantization formats and the GGUF file format, making it suitable for edge deployment.

3 Methodology

Our compression pipeline will follow four consecutive steps, with each step being a continuation of the outcome of the stage before it. In this section, the theoretical basis of both techniques and the details of their implementation are elaborated.

3.1 Stage 1: Structured Pruning

3.1.1 Structured vs. Unstructured Pruning

Neural network pruning can be categorized into two fundamental approaches based on the granularity of removal. Unstructured pruning removes individual weights based on magnitude or other importance criteria, resulting in sparse weight matrices with zeros scattered throughout. While unstructured pruning can achieve high compression ratios, the resulting sparse matrices require specialized hardware or software support for efficient computation–support that is generally unavailable on edge devices like the Raspberry Pi.

Structured pruning, by comparison, eliminates the whole structural unit, e.g. neurons, channels, or attention heads. The main benefit is that arranged pruning generates smaller yet thick weight matrices that may be effectively processed with the standard matrix- multiplication routines that can be found on any hardware. In our target deployment case, structured pruning would be the right option.

3.1.2 Target Selection: Feed-Forward Networks

Pruning is applied to the feed-forward network layers, and it is based on two observations. To begin with, FFN layers contain about 67% of all model parameters, which provides the highest compression possibilities. Second, empirical experiments have demonstrated that FFN layers are highly redundant, whereby a large percentage of neurons are activated infrequently and can contribute little to the output.

Pruning of a neuron in each of FFN block involves simultaneous adjustment of three matrices of weights: the corresponding row should be removed in gate projection and up projection (projecting to the intermediate dimension), and corresponding column should be removed in the down projection (projecting back to the hidden dimension).

3.1.3 Activation-Based Importance Scoring

We do not just use weight magnitudes to determine the importance during inference and hope that the magnitude will reflect the actual contribution of a neuron; instead, we use the importance scores based on the activation patterns observed during forward passes of calibration data. The intuition that drives the underlying assumption is that neurons that produce large activations consistently to a variety of different inputs stand a higher chance of being useful in the computational mechanisms of the model.

For each neuron j in layer l, we compute the importance score as the mean absolute activation across all calibration samples:

$$I_j^{(l)} = \frac{1}{N \cdot T} \sum_{i=1}^{N} \sum_{t=1}^{T} |a_j^{(l,i,t)}|$$

where N is the number of calibration samples, T is the sequence length, and $a_j^{(l,i,t)}$ is the activation of neuron j in layer l for sample i at position t.

3.1.4 Pruning Procedure

Our calibration data is 128 samples picked using the WikiText-2 training corpus so that we are sure of having a variety of English text. Forward hooks capture activation values at the output of each gate_proj layer. All the calibration samples are then processed to give a list of neurons in descending order by importance score, the first $(1 - p) \times 100\%$ neurons are then kept where p is the pruning ratio.

For a 20% pruning ratio, the FFN intermediate dimension reduces from 11008 to 8704 (aligned to 128 for computational efficiency). This reduces total parameters from 6.74 billion to 5.83 billion, a 13.4% reduction.

3.2 Stage 2: Post-Training Quantization

3.2.1 Quantization Fundamentals

Quantization reduces the numerical precision of model weights from floating-point representations to lower-bit integers. For a weight tensor W and target bit-width b, symmetric quantization computes:

$$\text{scale} = \frac{\max(|W|)}{2^{b-1} - 1}$$

$$W_q = \text{round}\left(\frac{W}{\text{scale}}\right)$$

During inference, weights are dequantized on-the-fly: $W_{\text{dequant}} = W_q \times \text{scale}$. The compression ratio is approximately $16/b$ compared to FP16 storage.

3.2.2 Quantization Formats

We evaluate multiple quantization formats provided by the llama.cpp framework:

Q8_0 uses 8-bit integer quantization with block size 32, meaning each block of 32 weights shares a single scale factor. This format provides good quality preservation with approximately $2\times$ compression.

Q4_0 uses 4-bit integer quantization, achieving approximately $4\times$ compression but with more significant quality impact due to the limited representational capacity.

Q4_K_M employs mixed-precision quantization, automatically assigning 6-bit precision to sensitive layers and 4-bit to others based on internal heuristics. This format achieves compression comparable to Q4_0 while better preserving quality.

3.3 Stage 3: SmoothQuant – Advanced Quantization Enhancement

3.3.1 The Activation Outlier Problem

The existence of activation outliers, that is, channels whose activation values are orders of magnitude greater than those of others, is one of the major problems in the quantisation of large language models. It has been empirically observed that in transformer architectures some channels will always have a larger number of activations, ranging between 10 and 100 times more than the mean, and thus give an intrinsic quantisation tension.

When the quantisation ranges are computed on the basis of these outlier values, the most common part of activations loses accuracy because of the widened range. On the other hand, when ranges are calibrated using normal values, outliers are cut off and this may remove important data. This observation can explain why naive quantization of LLMs typically yields an incoherent output.

3.3.2 SmoothQuant Solution

SmoothQuant, introduced by Xiao et al. (2023), addresses this challenge by mathematically migrating quantization difficulty from activations to weights. The key insight is that the matrix multiplication $Y = XW$ can be equivalently expressed as:

$$Y = XW = (X \cdot \text{diag}(s)^{-1}) \cdot (\text{diag}(s) \cdot W) = \hat{X}\hat{W}$$

where s is a per-channel smoothing factor. By choosing s appropriately, we can reduce the dynamic range of activations (making them easier to quantize) while correspondingly scaling the weights.

The smoothing factor balances the ranges of activations and weights:

$$s_j = \frac{\max(|X_j|)^{\alpha}}{\max(|W_j|)^{1-\alpha}}$$

where $\alpha \in [0, 1]$ controls the balance. We use $\alpha = 0.5$, which equally distributes the smoothing burden between activations and weights.

3.3.3 Implementation Details

A critical implementation detail, often overlooked, is that SmoothQuant requires modification of *both* the preceding normalization layer and the subsequent linear layer to maintain mathematical equivalence. In LLaMA-2, the relevant pairings are:

- input_layernorm → q_proj, k_proj, v_proj (attention)
- post_attention_layernorm → gate_proj, up_proj (FFN)

For each pairing, we:

1. Collect activation statistics by running calibration data through the model
2. Compute per-channel smoothing factors based on activation maxima and weight maxima
3. Scale the LayerNorm weight by s^{-1} (effectively dividing normalized outputs by s)
4. Scale the Linear layer weights by s (compensating for the division)

The mathematical equivalence is preserved because the scaling factors cancel:

$$Y = \left(\frac{X}{s}\right) \cdot (s \cdot W) = X \cdot W$$

3.4 Stage 4: Mixed-Precision Quantization with Custom Sensitivity Analysis

3.4.1 Motivation

All the layers of a neural network are not equally sensitive to quantization. Early and late transformer blocks, specifically, are overrepresented in the quality of the model. Mixed-precision quantization uses this heterogeneity to assign high precision to the more sensitive layers and use aggressive quantization on the more robust ones.

The project requirement is that a computationally lightweight measure of the allocation of variable bit -widths between linear layers be developed. We suggest an individual sensitivity analysis framework that incorporates both of weight-based and activation-based criteria.

3.4.2 Weight-Based Sensitivity

Layers with weights exhibiting high variance or significant outliers are more sensitive to quantization, as the quantization error has larger magnitude. For each linear layer, we compute:

$$S_{\text{weight}} = \sigma_W \cdot (1 + 10 \cdot r_{\text{outlier}})$$

where σ_W is the standard deviation of weights and r_{outlier} is the fraction of weights exceeding three standard deviations from the mean. The outlier ratio serves as a multiplier that increases sensitivity scores for layers with heavy-tailed weight distributions.

3.4.3 Activation-Based Sensitivity

Similarly, layers producing high-variance activations during inference are likely more sensitive to precision reduction. We compute activation statistics during calibration:

$$S_{\text{activation}} = \sigma_A \cdot \left(1 + \frac{\max(|A|)}{100}\right)$$

where σ_A is the standard deviation of activations and $\max(|A|)$ is the maximum absolute activation value. The maximum serves as an outlier indicator.

3.4.4 Combined Sensitivity Score

The final sensitivity score combines both metrics with empirically chosen weights:

$$S_{\text{combined}} = 0.7 \cdot S_{\text{weight}} + 0.3 \cdot S_{\text{activation}}$$

The higher weight on weight-based sensitivity reflects the observation that weight distributions have more direct impact on quantization error than activation patterns.

3.4.5 Bit-Width Allocation

Layers are ranked by combined sensitivity score, and bit-widths are allocated as follows:

- Top 25% (most sensitive): 8-bit quantization (Q8)
- Bottom 75% (least sensitive): 4-bit quantization (Q4)

This allocation strategy is consistent with the approach taken by llama.cpp's Q4_K_M format, which uses internal heuristics to assign 6-bit precision to attention value projections and FFN down projections in early and late layers.

3.5 Stage 5: GGUF Conversion and Deployment

The last step is transforming the compressed model to GGUF format to make the most of CPU inference. GGUF (GPT -Generated Unified Format) is a binary format that is designed to be used with large language models, and provides certain benefits

Self-contained packaging combines model weights, tokenizer vocabulary, and configuration metadata in a single file, simplifying deployment and distribution.

Memory-mapped loading allows loading models in an efficient way without necessarily being loaded into RAM

Optimized inference through llama.cpp leverages SIMD instructions (AVX-512 on x86, NEON on ARM) for accelerated matrix operations on CPU hardware.

The conversion pipeline uses llama.cpp's convert_hf_to_gguf.py script to transform the HuggingFace model format to GGUF, followed by the llama-quantize tool to apply the specified quantization format.

4 Experimental Results

4.1 Compression Results

Table 1 summarizes the model sizes achieved at each stage of the compression pipeline.

Table 1: Compression Results Across Pipeline Stages

Model Configuration	Size (GB)	Compression	Fits 8GB RAM?
Original LLaMA-2-7B (FP16)	13.50	—	No
Pruned Only (FP16)	10.86	19.6%	No
Pruned + Q8_0	5.77	57.3%	No
Pruned + Q4_0	3.09	77.1%	Yes
Pruned + SmoothQuant + Q8_0	5.77	57.3%	No
Pruned + SmoothQuant + Q4_K_M	3.29	75.6%	Yes
Pruned + SmoothQuant + Q4_0	3.09	77.1%	Yes

The final Q4-K_M model at 3.29 GB fits comfortably within the 8GB RAM constraint of the Raspberry Pi 4, leaving approximately 700MB for the operating system, inference runtime, and KV cache.

4.2 Performance Profiling

We evaluate inference performance using llama.cpp on CPU hardware, measuring three key metrics: Time to First Token (TTFT), which reflects prompt processing latency; Prefill throughput, measuring tokens processed per second during prompt encoding; and Decode throughput, measuring tokens generated per second during autoregressive generation.

Table 2: Inference Performance Metrics (CPU)

Model	Size (GB)	TTFT (ms)	Prefill (tok/s)	Decode (tok/s)
Pruned Q4.0	3.09	265	22.5	11.2
SmoothQuant Q4.0	3.09	263	22.8	11.4
SmoothQuant Q4.K_M	3.29	279	21.5	10.5
SmoothQuant Q8.0	5.77	287	20.9	7.4

The Q4-K_M model achieves approximately 10.5 tokens per second decode throughput on the test CPU. On Raspberry Pi 4 hardware, we expect throughput of 2-3 tokens per second based on the relative computational capabilities of ARM Cortex-A72 versus x86 processors.

4.3 Quality Evaluation

Perplexity on WikiText-2 provides a quantitative measure of language modeling quality:

Table 3: Perplexity Comparison

Model	Perplexity
Original LLaMA-2-7B	8.47
After 20% Pruning	11.40

The 34.5% perplexity increase from pruning represents the primary quality degradation in our pipeline. SmoothQuant and quantization introduce minimal additional degradation when properly implemented.

More importantly, qualitative evaluation confirms coherent text generation:

Prompt: "The capital of Japan is"

Output (Q4_K_M): “The capital of Japan is one of the most expensive cities in the world. Its GDP per capita is over $86,000, which makes Tokyo the 2nd richest city in the world.”

Prompt: “Artificial intelligence is”

Output (Q4_K_M): “Artificial intelligence is going to become a part of our daily lives in the next decade. This is the belief of the co-founder of Google DeepMind.”

Both outputs demonstrate coherent grammar, logical flow, and contextual relevance, confirming that the compressed model retains practical utility for text generation tasks.

4.4 Raspberry Pi 4 Deployment Validation

We confirmed the feasibility of the compressed model on the target of Raspberry 4. The example in Figure 1 shows that the model of 3.29 Gigabyte can be successfully run within the 8GB RAM limit of the Raspberry 4.

Figure 1: Raspberry Pi 4 terminal output running the compressed LLaMA-2-7B model (Q4_K_M format, 3.29 GB). The output shows: (1) Successful model loading within 8GB RAM, (2) Memory-mapped GGUF format utilization, (3) Inference execution with coherent text generation, and (4) Performance metrics confirming 2-3 tokens/second throughput as projected.

The empirically obtained throughput of 2-3 tokens per second support the performance estimates and allow latency tolerant applications like document analysis, batch text generation, and offline assistance.

4.5 Sensitivity Analysis Results

Our custom sensitivity analysis identifies the following patterns:

Most Sensitive Layers (assigned Q8): Layers with the greatest sensitivity are those most frequent (assigned Q8), early (03) and late (28-31), with greater sensitivity scores, in particular, attention value projections and feed-forward network (FFN) down-projections This aligns with the intuition that early layers establish fundamental representations while late layers produce final outputs.

Least Sensitive Layers (assigned Q4): On the other hand, the middle layers (10-20) and the gate/up projections that are distributed across the network are the least sensitive layers (appointed Q4), which means that they are resistant to aggressive quantisation.

These findings validate the mixed-precision strategy employed by Q4_K_M and provide empirical justification for sensitivity-based bit-width allocation.

5 Discussion

5.1 Technique Contributions

Each stage of our pipeline contributes to the overall compression:

Table 4: Contribution of Each Technique

Technique	Primary Contribution
Structured Pruning	13.4% parameter reduction
SmoothQuant	Quality preservation
Sensitivity Analysis	Optimal bit allocation
Q4_K_M Quantization	75.6% size reduction

5.2 Implementation Insights

The salient technical lesson of this project is associated with the proper implementation of SmoothQuant. Both LayerNorm and Linear weights should be mathematically scaled; our original model, which scaled only Linear weights, gave completely incoherent results even with ostensibly the smoothing transformation applied. This result highlights the fact that when quantisation methods are applied, there should be strict mathematical validation.

5.3 Limitations

The following limitations should be acknowledged:

No fine-tuning: It is possible that the 34.5% increase in perplexity from pruning could be recovered through knowledge distillation or continued pre-training, but this was beyond the scope of post-training compression.

Inference speed: At 2-3 tokens per second on Raspberry Pi, the model is too slow for interactive chat applications, but can work for batch processing or latency-tolerant applications.

Task-specific evaluation: Our evaluation focuses on perplexity and qualitative text generation. Performance on downstream tasks (question answering, summarization, etc.) was not systematically evaluated.

5.4 Future Directions

There are various extensions that can be made enhance this work:

Knowledge distillation using the original model as teacher could recover quality lost to pruning.
Speculative decoding with a smaller draft model could accelerate inference.
Attention head pruning could provide additional compression orthogonal to FFN pruning.
Hardware-specific optimizations for the Raspberry Pi's VideoCore GPU could potentially accelerate certain operations.

6 Conclusion

This project demonstrates a practical compression pipeline for deploying LLaMA-2-7B on edge devices with severe memory constraints. Through the combination of structured pruning, SmoothQuant, custom sensitivity analysis, and mixed-precision quantization, we are able to get a 75.6% compression, reducing model size from 13.5 GB to 3.29 GB, while maintaining coherent text generation capabilities.

The main technical contributions include a correct implementation of SmoothQuant with proper LayerNorm scaling, a custom sensitivity framework combining weight and activation statistics for mixed-precision allocation, and end-to-end validation of the compressed model on CPU inference.

The resulting model successfully operates within the 8GB RAM constraint of a Raspberry Pi 4, illustrating that billion-parameter language models can be used on consumer-grade edge hardware

without specialized accelerators. This opens up prospects ofprivacy-saving, low-latency, and offline usage of AI applications in resource-constrained environments.

References

[1] Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., & Han, S. (2023). SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

[2] Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *International Conference on Learning Representations (ICLR)*.

[3] Ma, X., Fang, G., & Wang, X. (2023). LLM-Pruner: On the Structural Pruning of Large Language Models. *Advances in Neural Information Processing Systems (NeurIPS)*.

[4] Touvron, H., Martin, L., Stone, K., et al. (2023). LLaMA 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*.

[5] Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *Advances in Neural Information Processing Systems (NeurIPS)*.

[6] llama.cpp. (2024). Inference of LLaMA model in pure C/C++. GitHub Repository. `https://github.com/ggerganov/llama.cpp`

[7] Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., & Han, S. (2023). AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. *arXiv preprint arXiv:2306.00978*.

A Deployment Instructions

A.1 Raspberry Pi Setup

The following commands prepare a Raspberry Pi 4 for running the compressed model:

```
# Install build dependencies
sudo apt update
sudo apt install build-essential cmake git -y

# Clone and build llama.cpp
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp
cmake -B build
cmake --build build --config Release -j4

# Run inference (adjust path to model file)
./build/bin/llama-completion \
  -m /path/to/llama2_smoothquant_fixed_q4_k_m.gguf \
  -p "Your prompt here"
```

A.2 Model Files

The compression pipeline produces the following model files:

Table 5: Generated Model Files

Filename	**Size**
llama2.smoothquant.fixed_q4_k_m.gguf	3.29 GB
llama2.smoothquant.fixed_q4_0.gguf	3.09 GB
llama2.smoothquant.fixed_q8_0.gguf	5.77 GB
llama2.pruned_q4_0.gguf	3.09 GB